Preprocess الأنظمة (Baseline legal layer)

```
#!/usr/bin/env python3
"""
Arabic Legal Baseline Dataset Builder
=====================================

Builds a reproducible baseline dataset from raw Arabic legal files
(JSON / JSONL / CSV / XLSX), with a focus on Saudi legal system files.

Key capabilities:
- Accepts a ZIP archive or extracted folder as input.
- Recursively loads supported files.
- Normalizes heterogeneous schemas into one article-level dataset.
- Preserves Arabic legal meaning and hierarchy.
- Filters inactive / deleted / repealed articles.
- Prefers the latest valid text when modifications exist.
- Exports merged, filtered, excluded, problematic, and QA reports.

Allowed libraries only:
- pandas
- numpy
- pathlib
- json
- zipfile
- tempfile
- shutil
- re
- uuid
- logging
- openpyxl (via pandas for xlsx)
"""

from __future__ import annotations

import json
import logging
import re
import shutil
import tempfile
import uuid
import zipfile
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Tuple

import numpy as np
import pandas as pd
```

```
# ---------------------------------------------------------------------------
# Configuration
# ---------------------------------------------------------------------------

SUPPORTED_EXTENSIONS = {".json", ".jsonl", ".csv", ".xlsx"}

STANDARD_COLUMNS = [
    "record_id",
    "law_id",
    "law_name",
    "law_info",
    "law_brief",
    "chapter_name",
    "article_title",
    "article_number",
    "article_text_raw",
    "article_text_clean",
    "original_text",
    "current_text",
    "article_status",
    "is_modified",
    "is_active",
    "modification_decree",
    "modification_details",
    "text_source_used",
    "source_file",
    "source_format",
    "hierarchy_level",
    "hierarchy_depth",
```

```
    "text_length_chars",
    "word_count",
    "has_article_number",
]

INACTIVE_STATUS_TERMS = {
    "ملغاة",
    "ملغي",
    "محذوفة",
    "محذوف",
    "غير سارية",
    "منسوخة",
    "منسوخ",
    "موقوفة",
    "موقوف",
    "ملغى",
    "لا يعمل بها",
    "متروكة",
    "منتهية",
    "منتهي",
}

INACTIVE_TEXT_PATTERNS = [
    r"\b(?:ملغ(اة|ى|ي)\b",
    r"\b(?:محذوف(ة)?\b",
    r"\bغير\s+سارية\b",
    r"\b(?:منسوخ(ة)?\b",
    r"\b(?:موقوف(ة)?\b",
    r"\bلا\s+يعمل\s+بها\b",
]

SHORT_ARTICLE_THRESHOLD = 15
DEFAULT_OUTPUT_DIRNAME = "output"
```

```
# ---------------------------------------------------------------------------
# Logging
# ---------------------------------------------------------------------------


def setup_logging(output_dir: Path) -> logging.Logger:
    """Create console + file logger."""
    output_dir.mkdir(parents=True, exist_ok=True)
    log_file = output_dir / "pipeline.log"

    logger = logging.getLogger("arabic_legal_pipeline")
    logger.setLevel(logging.INFO)
    logger.handlers.clear()

    formatter = logging.Formatter(
        "%(asctime)s | %(levelname)s | %(message)s", "%Y-%m-%d %H:%M:%S"
    )

    stream_handler = logging.StreamHandler()
    stream_handler.setFormatter(formatter)

    file_handler = logging.FileHandler(log_file, encoding="utf-8")
    file_handler.setFormatter(formatter)

    logger.addHandler(stream_handler)
    logger.addHandler(file_handler)
    return logger
```

```
# ---------------------------------------------------------------------------
# Utility helpers
# ---------------------------------------------------------------------------


def safe_str(value: Any) -> Optional[str]:
    """Return stripped string or None."""
    if value is None:
        return None
    if isinstance(value, float) and pd.isna(value):
        return None
    text = str(value).strip()
    return text if text else None



def first_non_empty(*values: Any) -> Optional[str]:
    """Return the first non-empty value as a string."""
```

```
    for value in values:
        cleaned = safe_str(value)
        if cleaned:
            return cleaned
    return None



def bool_from_any(value: Any) -> Optional[bool]:
    """Normalize booleans from mixed raw values."""
    if value is None:
        return None
    if isinstance(value, bool):
        return value
    if isinstance(value, (int, np.integer)):
        return bool(value)

    text = str(value).strip().lower()
    if text in {"true", "1", "yes", "y", "نعم", "صح"}:
        return True
    if text in {"false", "0", "no", "n", "لا", "خطأ", "خطا"}:
        return False
    return None



def infer_source_format(file_path: Path) -> str:
    """Infer source format from file suffix."""
    return file_path.suffix.lower().lstrip(".")



def extract_input_to_workdir(input_path: Path, logger: logging.Logger) -> Tuple[Path, Optional[tempfile.Temporar
    """
    Return the working directory.
    - If input is ZIP, extract to temp dir.
    - If input is folder, use directly.
    """
    if not input_path.exists():
        raise FileNotFoundError(f"Input path does not exist: {input_path}")

    if input_path.is_file() and input_path.suffix.lower() == ".zip":
        temp_dir = tempfile.TemporaryDirectory(prefix="arabic_legal_pipeline_")
        workdir = Path(temp_dir.name)
        logger.info("Extracting ZIP input: %s -> %s", input_path, workdir)
        with zipfile.ZipFile(input_path, "r") as zf:
            zf.extractall(workdir)
        return workdir, temp_dir

    if input_path.is_dir():
        logger.info("Using extracted folder input: %s", input_path)
        return input_path, None

    raise ValueError("Input path must be a ZIP archive or a directory.")



def discover_files(root_dir: Path) -> List[Path]:
    """Recursively find supported files."""
    return sorted(
        p for p in root_dir.rglob("*") if p.is_file() and p.suffix.lower() in SUPPORTED_EXTENSIONS
    )
```

```
# ------------------------------------------------------------------------------
# Arabic legal text normalization
# ------------------------------------------------------------------------------


def remove_diacritics(text: str) -> str:
    """Remove Arabic diacritics only."""
    arabic_diacritics = re.compile(
        r"[\u0610-\u061A\u064B-\u065F\u0670\u06D6-\u06ED]"
    )
    return arabic_diacritics.sub("", text)



def normalize_arabic_text(
    text: Optional[str],
    *,
    strip_diacritics: bool = True,
) -> Optional[str]:
```

```
    """
    Carefully normalize Arabic legal text while preserving legal meaning.

    Keeps:
    - numbers
    - article references
    - legal terminology
    - punctuation itself (only spacing is normalized)
    """
    if text is None:
        return None

    text = str(text)
    if not text.strip():
        return None

    # Remove tatweel
    text = text.replace("_", "")

    # Normalize Alef variants
    text = re.sub(r"[ٱإأآا]", "ا", text)

    # Normalize Yeh / Alef Maqsura carefully
    text = text.replace("ى", "ي")

    # Normalize Teh Marbuta is NOT changed intentionally to preserve wording.
    # Hamza standalone is not altered.

    if strip_diacritics:
        text = remove_diacritics(text)

    # Normalize Arabic / Latin whitespace
    text = re.sub(r"[\t\r\f\v]+", " ", text)
    text = re.sub(r"\n+", " ", text)
    text = re.sub(r"\s+", " ", text)

    # Normalize punctuation spacing without removing punctuation
    # Remove spaces before punctuation
    text = re.sub(r"\s+([\.,،؛:!?\)\]\}])", r"\1", text)
    # Ensure one space after punctuation if followed by a word/number and not already spaced
    text = re.sub(r"([\.,،؛:!?])([^\s\.,،؛:!?\)\]\}])", r"\1 \2", text)
    # Remove spaces after opening brackets
    text = re.sub(r"([\(\[\{])\s+", r"\1", text)
    # Remove duplicate spaces again
    text = re.sub(r"\s{2,}", " ", text)

    return text.strip()
```

```
# ------------------------------------------------------------------------------
# Status / activity logic
# ------------------------------------------------------------------------------


def normalize_status(status: Optional[str]) -> Optional[str]:
    """Normalize raw status string lightly for comparison."""
    if status is None:
        return None
    status = normalize_arabic_text(status, strip_diacritics=True)
    return status



def text_indicates_inactive(text: Optional[str]) -> bool:
    """Check inactive/repealed markers inside text."""
    if not text:
        return False
    normalized = normalize_arabic_text(text, strip_diacritics=True) or ""
    for pattern in INACTIVE_TEXT_PATTERNS:
        if re.search(pattern, normalized):
            return True
    return False



def determine_activity(record: Dict[str, Any]) -> bool:
    """
    Determine whether article is active/current.

    Uses status first, then modification/details signals.
    """
    status = normalize_status(record.get("article_status"))
    modification_details = record.get("modification_details")
```

```
    current_text = record.get("current_text")

    if status:
        for term in INACTIVE_STATUS_TERMS:
            if term in status:
                return False

    if text_indicates_inactive(modification_details):
        return False

    if text_indicates_inactive(current_text):
        return False

    return True



def choose_current_text(article: Dict[str, Any]) -> Tuple[Optional[str], Optional[str]]:
    """
    Choose the best current valid article text.

    Priority:
    1) newArticleText if modified and valid
    2) articleText
    3) originalText

    Returns:
        (chosen_text, text_source_used)
    """
    is_modified = bool_from_any(article.get("isModified"))
    new_text = safe_str(article.get("newArticleText"))
    article_text = safe_str(article.get("articleText"))
    original_text = safe_str(article.get("originalText"))

    if is_modified and new_text and not text_indicates_inactive(new_text):
        return new_text, "newArticleText"

    if article_text and not text_indicates_inactive(article_text):
        return article_text, "articleText"

    if original_text and not text_indicates_inactive(original_text):
        return original_text, "originalText"

    return None, None
```

```
# ------------------------------------------------------------------------------
# Schema normalization and parsing
# ------------------------------------------------------------------------------


def compute_hierarchy_depth(chapter_name: Optional[str], hierarchy_level: Optional[str]) -> int:
    """Compute a simple hierarchy depth."""
    depth = 1  # article level always exists
    if safe_str(chapter_name):
        depth += 1
    if safe_str(hierarchy_level) and hierarchy_level != "article":
        depth += 1
    return depth



def build_stable_record_id(law_id: Any, law_name: Any, article_number: Any, current_text: Any) -> str:
    """Build stable deterministic UUID5 record ID."""
    namespace = uuid.UUID("12345678-1234-5678-1234-567812345678")
    signature = "||".join(
        [
            safe_str(law_id) or "",
            safe_str(law_name) or "",
            safe_str(article_number) or "",
            safe_str(current_text) or "",
        ]
    )
    return str(uuid.uuid5(namespace, signature))



def normalize_article_record(
    law_level: Dict[str, Any],
    article: Dict[str, Any],
    source_file: Path,
) -> Dict[str, Any]:
    """Normalize a single raw article into the standard schema."""
```

```
    current_text, text_source_used = choose_current_text(article)
    article_text_raw = first_non_empty(article.get("articleText"), article.get("newArticleText"), article.get("o
    current_text_clean = normalize_arabic_text(current_text, strip_diacritics=True)

    law_id = first_non_empty(law_level.get("lawId"), law_level.get("law_id"))
    law_name = first_non_empty(law_level.get("lawName"), law_level.get("law_name"))
    chapter_name = first_non_empty(article.get("chapter"), article.get("chapter_name"))
    article_title = first_non_empty(article.get("articleTitle"), article.get("article_title"))
    article_number = first_non_empty(article.get("articleNumber"), article.get("article_number"))
    article_status = first_non_empty(article.get("status"), article.get("article_status"))
    is_modified = bool_from_any(article.get("isModified"))
    original_text = safe_str(article.get("originalText"))
    modification_decree = safe_str(article.get("modificationDecree"))
    modification_details = safe_str(article.get("modificationDetails"))

    hierarchy_level = "article"
    hierarchy_depth = compute_hierarchy_depth(chapter_name, hierarchy_level)

    record_id = build_stable_record_id(law_id, law_name, article_number, current_text)

    record = {
        "record_id": record_id,
        "law_id": law_id,
        "law_name": law_name,
        "law_info": safe_str(law_level.get("info")),
        "law_brief": safe_str(law_level.get("brief")),
        "chapter_name": chapter_name,
        "article_title": article_title,
        "article_number": article_number,
        "article_text_raw": article_text_raw,
        "article_text_clean": current_text_clean,
        "original_text": original_text,
        "current_text": current_text,
        "article_status": article_status,
        "is_modified": is_modified,
        "is_active": None,  # computed next
        "modification_decree": modification_decree,
        "modification_details": modification_details,
        "text_source_used": text_source_used,
        "source_file": str(source_file),
        "source_format": infer_source_format(source_file),
        "hierarchy_level": hierarchy_level,
        "hierarchy_depth": hierarchy_depth,
        "text_length_chars": len(current_text_clean) if current_text_clean else 0,
        "word_count": len(current_text_clean.split()) if current_text_clean else 0,
        "has_article_number": article_number is not None,
    }
    record["is_active"] = determine_activity(record)
    return record



def parse_law_dict(data: Dict[str, Any], source_file: Path) -> List[Dict[str, Any]]:
    """Parse the common raw law JSON dict structure with articles[]."""
    records: List[Dict[str, Any]] = []
    articles = data.get("articles")

    if not isinstance(articles, list):
        return records

    for article in articles:
        if not isinstance(article, dict):
            continue
        record = normalize_article_record(data, article, source_file)
        records.append(record)

    return records



def normalize_flat_dataframe(df: pd.DataFrame, source_file: Path) -> List[Dict[str, Any]]:
    """
    Normalize a dataframe if rows are already article-like.
    Useful for CSV/XLSX/JSONL sources.
    """
    records: List[Dict[str, Any]] = []
    if df.empty:
        return records

    for row in df.to_dict(orient="records"):
        row = {k: v for k, v in row.items()}
        law_level = {
```

```
            "lawId": row.get("lawId") or row.get("law_id"),
            "lawName": row.get("lawName") or row.get("law_name"),
            "info": row.get("info") or row.get("law_info"),
            "brief": row.get("brief") or row.get("law_brief"),
        }
        article = {
            "chapter": row.get("chapter") or row.get("chapter_name"),
            "articleTitle": row.get("articleTitle") or row.get("article_title"),
            "articleNumber": row.get("articleNumber") or row.get("article_number"),
            "articleText": row.get("articleText") or row.get("article_text") or row.get("article_text_raw"),
            "originalText": row.get("originalText") or row.get("original_text"),
            "newArticleText": row.get("newArticleText") or row.get("new_article_text") or row.get("current_text"
            "status": row.get("status") or row.get("article_status"),
            "isModified": row.get("isModified") or row.get("is_modified"),
            "modificationDecree": row.get("modificationDecree") or row.get("modification_decree"),
            "modificationDetails": row.get("modificationDetails") or row.get("modification_details"),
        }
        records.append(normalize_article_record(law_level, article, source_file))
    return records
```

```
# ------------------------------------------------------------------------------
# File readers
# ------------------------------------------------------------------------------


def read_json_file(file_path: Path) -> Any:
    """Read a UTF-8 JSON file."""
    with open(file_path, "r", encoding="utf-8") as f:
        return json.load(f)



def read_jsonl_file(file_path: Path) -> pd.DataFrame:
    """Read a UTF-8 JSONL file into a dataframe."""
    rows = []
    with open(file_path, "r", encoding="utf-8") as f:
        for line_number, line in enumerate(f, start=1):
            line = line.strip()
            if not line:
                continue
            try:
                rows.append(json.loads(line))
            except Exception as exc:
                raise ValueError(f"Invalid JSONL at line {line_number}: {exc}") from exc
    return pd.DataFrame(rows)



def load_supported_file(file_path: Path) -> Tuple[List[Dict[str, Any]], Optional[str]]:
    """
    Load one file and normalize records.

    Returns:
        (records, error_message)
    """
    try:
        suffix = file_path.suffix.lower()

        if suffix == ".json":
            data = read_json_file(file_path)
            if isinstance(data, dict):
                return parse_law_dict(data, file_path), None
            if isinstance(data, list):
                df = pd.DataFrame(data)
                return normalize_flat_dataframe(df, file_path), None
            return [], f"Unsupported JSON root type: {type(data)}"

        if suffix == ".jsonl":
            df = read_jsonl_file(file_path)
            return normalize_flat_dataframe(df, file_path), None

        if suffix == ".csv":
            df = pd.read_csv(file_path, encoding="utf-8")
            return normalize_flat_dataframe(df, file_path), None

        if suffix == ".xlsx":
            df = pd.read_excel(file_path, engine="openpyxl")
            return normalize_flat_dataframe(df, file_path), None

        return [], f"Unsupported file extension: {suffix}"

    except Exception as exc:
```

```
        return [], str(exc)
```

```
# ---------------------------------------------------------------------------
# Quality checks, filtering, and reporting
# ---------------------------------------------------------------------------


def ensure_standard_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Add missing standard columns with nulls and order them."""
    for col in STANDARD_COLUMNS:
        if col not in df.columns:
            df[col] = None
    return df[STANDARD_COLUMNS]



def detect_problematic_rows(df: pd.DataFrame) -> pd.DataFrame:
    """Collect suspicious/problematic rows for review."""
    problems: List[pd.DataFrame] = []

    empty_clean = df[df["article_text_clean"].isna() | (df["article_text_clean"].astype(str).str.strip() == "")]
    if not empty_clean.empty:
        empty_clean["problem_type"] = "empty_cleaned_text"
        problems.append(empty_clean)

    short_articles = df[
        df["article_text_clean"].fillna("").str.len().between(1, SHORT_ARTICLE_THRESHOLD)
    ].copy()
    if not short_articles.empty:
        short_articles["problem_type"] = "suspiciously_short_article"
        problems.append(short_articles)

    repeated_numbers = df[
        df["law_name"].notna() & df["article_number"].notna()
    ].copy()
    if not repeated_numbers.empty:
        repeated_numbers["dup_count"] = repeated_numbers.groupby(["law_name", "article_number"])["record_id"].tr
        repeated_numbers = repeated_numbers[repeated_numbers["dup_count"] > 1].copy()
        if not repeated_numbers.empty:
            repeated_numbers["problem_type"] = "repeated_article_number_within_law"
            problems.append(repeated_numbers)

    duplicated_ids = df[df["record_id"].duplicated(keep=False)].copy()
    if not duplicated_ids.empty:
        duplicated_ids["problem_type"] = "duplicate_record_id"
        problems.append(duplicated_ids)

    if not problems:
        return pd.DataFrame(columns=list(df.columns) + ["problem_type"])

    problematic = pd.concat(problems, ignore_index=True)
    problematic = problematic.drop_duplicates(subset=["record_id", "problem_type"])
    return problematic



def deduplicate_articles(df: pd.DataFrame) -> Tuple[pd.DataFrame, Dict[str, int]]:
    """Remove exact duplicates carefully."""
    stats = {
        "duplicates_removed_by_law_article_text": 0,
        "duplicates_removed_by_exact_text": 0,
    }

    before = len(df)
    df = df.drop_duplicates(subset=["law_name", "article_number", "current_text"], keep="first").copy()
    stats["duplicates_removed_by_law_article_text"] = before - len(df)

    before2 = len(df)
    df = df.drop_duplicates(subset=["current_text"], keep="first").copy()
    stats["duplicates_removed_by_exact_text"] = before2 - len(df)

    return df, stats



def summarize_missing_values(df: pd.DataFrame) -> Dict[str, int]:
    """Return missing-value summary per column."""
    return {col: int(df[col].isna().sum()) for col in df.columns}
```

```
def dataframe_to_jsonl(df: pd.DataFrame, output_path: Path) -> None:
    """Write dataframe to UTF-8 JSONL."""
    with open(output_path, "w", encoding="utf-8") as f:
        for record in df.to_dict(orient="records"):
            f.write(json.dumps(record, ensure_ascii=False) + "\n")



def save_outputs(
    raw_df: pd.DataFrame,
    active_df: pd.DataFrame,
    excluded_df: pd.DataFrame,
    problematic_df: pd.DataFrame,
    report: Dict[str, Any],
    output_dir: Path,
) -> None:
    """Persist all required outputs."""
    output_dir.mkdir(parents=True, exist_ok=True)

    raw_df.to_csv(output_dir / "raw_merged_articles.csv", index=False, encoding="utf-8-sig")
    active_df.to_csv(output_dir / "baseline_active_articles.csv", index=False, encoding="utf-8-sig")
    dataframe_to_jsonl(active_df, output_dir / "baseline_active_articles.jsonl")
    excluded_df.to_csv(output_dir / "excluded_inactive_articles.csv", index=False, encoding="utf-8-sig")
    problematic_df.to_csv(output_dir / "problematic_rows.csv", index=False, encoding="utf-8-sig")

    with open(output_dir / "data_quality_report.json", "w", encoding="utf-8") as f:
        json.dump(report, f, ensure_ascii=False, indent=2)
```

```
# ---------------------------------------------------------------------------
# Main pipeline
# ---------------------------------------------------------------------------


def build_baseline_dataset(input_path: Path, output_dir: Path) -> Dict[str, Any]:
    """Run the full pipeline and return the report."""
    logger = setup_logging(output_dir)
    logger.info("Starting Arabic legal baseline pipeline")
    logger.info("Input path: %s", input_path)
    logger.info("Output dir: %s", output_dir)

    parse_failures: List[Dict[str, str]] = []
    all_records: List[Dict[str, Any]] = []
    temp_dir_handle: Optional[tempfile.TemporaryDirectory] = None

    try:
        workdir, temp_dir_handle = extract_input_to_workdir(input_path, logger)
        files = discover_files(workdir)
        logger.info("Discovered %d supported input files", len(files))

        for file_path in files:
            logger.info("Loading file: %s", file_path)
            records, error = load_supported_file(file_path)
            if error:
                logger.warning("Failed parsing file: %s | %s", file_path, error)
                parse_failures.append({"file": str(file_path), "error": error})
                continue

            logger.info("Extracted %d records from %s", len(records), file_path.name)
            all_records.extend(records)

        raw_df = pd.DataFrame(all_records)
        if raw_df.empty:
            logger.warning("No article records were extracted.")
            raw_df = pd.DataFrame(columns=STANDARD_COLUMNS)
        else:
            raw_df = ensure_standard_columns(raw_df)

        # Quality checks on raw merged data
        problematic_df = detect_problematic_rows(raw_df)

        # Keep rows with usable current_text only
        usable_df = raw_df[
            raw_df["current_text"].notna() & (raw_df["current_text"].astype(str).str.strip() != "")
        ].copy()

        # Preserve rows with missing metadata if text exists; drop only no-text rows
        excluded_no_text_df = raw_df[
            raw_df["current_text"].isna() | (raw_df["current_text"].astype(str).str.strip() == "")
        ].copy()
        if not excluded_no_text_df.empty:
```

```
            excluded_no_text_df["exclusion_reason"] = "no_usable_article_text"

        active_df = usable_df[usable_df["is_active"] == True].copy()  # noqa: E712
        inactive_df = usable_df[usable_df["is_active"] == False].copy()  # noqa: E712
        if not inactive_df.empty:
            inactive_df["exclusion_reason"] = "inactive_deleted_repealed_or_not_current"

        before_dedup = len(active_df)
        active_df, dedup_stats = deduplicate_articles(active_df)
        duplicates_removed_total = before_dedup - len(active_df)

        excluded_df = pd.concat([inactive_df, excluded_no_text_df], ignore_index=True) if not (inactive_df.empty

        # Recompute / validate stable IDs after dedup (optional sanity)
        if not active_df.empty:
            active_df["record_id_recomputed"] = active_df.apply(
                lambda row: build_stable_record_id(
                    row["law_id"], row["law_name"], row["article_number"], row["current_text"]
                ),
                axis=1,
            )
            id_mismatch_count = int((active_df["record_id"] != active_df["record_id_recomputed"]).sum())
            active_df = active_df.drop(columns=["record_id_recomputed"])
        else:
            id_mismatch_count = 0

        laws_processed = int(active_df["law_name"].dropna().nunique()) if not active_df.empty else 0
        total_modified_articles = int(raw_df["is_modified"].fillna(False).astype(bool).sum()) if not raw_df.empt
        used_new_article_text_count = int((raw_df["text_source_used"] == "newArticleText").sum()) if not raw_df.
        average_text_length = float(active_df["text_length_chars"].mean()) if not active_df.empty else 0.0

        report = {
            "input_summary": {
                "input_path": str(input_path),
                "number_of_input_files": len(files) if 'files' in locals() else 0,
                "supported_extensions": sorted(SUPPORTED_EXTENSIONS),
            },
            "processing_summary": {
                "number_of_laws_processed": laws_processed,
                "total_articles_extracted": int(len(raw_df)),
                "total_active_articles_kept": int(len(active_df)),
                "total_inactive_deleted_repealed_removed": int(len(inactive_df)),
                "total_rows_removed_no_usable_text": int(len(excluded_no_text_df)),
                "total_modified_articles": total_modified_articles,
                "articles_where_newArticleText_was_used": used_new_article_text_count,
                "duplicates_removed_total": duplicates_removed_total,
                **dedup_stats,
                "stable_record_id_mismatches": id_mismatch_count,
                "average_text_length_chars_active": average_text_length,
            },
            "validation_summary": {
                "suspiciously_short_articles": int(
                    (raw_df["article_text_clean"].fillna("").str.len().between(1, SHORT_ARTICLE_THRESHOLD)).sum(
                ) if not raw_df.empty else 0,
                "empty_cleaned_text_rows": int(
                    (raw_df["article_text_clean"].isna() | (raw_df["article_text_clean"].astype(str).str.strip()
                ) if not raw_df.empty else 0,
                "repeated_article_numbers_within_same_law": int(
                    raw_df[
                        raw_df["law_name"].notna() & raw_df["article_number"].notna()
                    ].groupby(["law_name", "article_number"]).size().gt(1).sum()
                ) if not raw_df.empty else 0,
            },
            "missing_values_summary": summarize_missing_values(raw_df) if not raw_df.empty else {},
            "files_failed_to_parse": parse_failures,
        }

        save_outputs(raw_df, active_df, excluded_df, problematic_df, report, output_dir)
        logger.info("Pipeline completed successfully")
        logger.info("Active articles kept: %d", len(active_df))
        logger.info("Excluded inactive articles: %d", len(inactive_df))
        logger.info("Problematic rows flagged: %d", len(problematic_df))
        return report

    finally:
        if temp_dir_handle is not None:
            temp_dir_handle.cleanup()
```

```
# ------------------------------------------------------------------------------
# CLI entry point
# ------------------------------------------------------------------------------
```

```
def main() -> None:
    """
    Example usage:
        python arabic_legal_baseline_pipeline.py

    Edit INPUT_PATH and OUTPUT_DIR below as needed.
    """
    INPUT_PATH = Path("/content/laws_raw.zip")
    OUTPUT_DIR = Path(DEFAULT_OUTPUT_DIRNAME)

    report = build_baseline_dataset(INPUT_PATH, OUTPUT_DIR)
    print(json.dumps(report, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

```
2026-04-13 17:08:52 | INFO | Starting Arabic legal baseline pipeline
INFO:arabic_legal_pipeline:Starting Arabic legal baseline pipeline
2026-04-13 17:08:52 | INFO | Input path: /content/laws_raw.zip
INFO:arabic_legal_pipeline:Input path: /content/laws_raw.zip
2026-04-13 17:08:52 | INFO | Output dir: output
INFO:arabic_legal_pipeline:Output dir: output
2026-04-13 17:08:52 | INFO | Extracting ZIP input: /content/laws_raw.zip -> /tmp/arabic_legal_pipeline_2hioi8un
INFO:arabic_legal_pipeline:Extracting ZIP input: /content/laws_raw.zip -> /tmp/arabic_legal_pipeline_2hioi8un
2026-04-13 17:08:52 | INFO | Discovered 25 supported input files
INFO:arabic_legal_pipeline:Discovered 25 supported input files
2026-04-13 17:08:52 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الأسماء التجارية.json
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الأسماء التجارية.json
2026-04-13 17:08:52 | INFO | Extracted 23 records from نظام الأسماء التجارية.json
INFO:arabic_legal_pipeline:Extracted 23 records from نظام الأسماء التجارية.json
2026-04-13 17:08:52 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الأوراق التجارية.json
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الأوراق التجارية.json
2026-04-13 17:08:52 | INFO | Extracted 121 records from نظام الأوراق التجارية.json
INFO:arabic_legal_pipeline:Extracted 121 records from نظام الأوراق التجارية.json
2026-04-13 17:08:52 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الإثبات.json
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الإثبات.json
2026-04-13 17:08:52 | INFO | Extracted 129 records from نظام الإثبات.json
INFO:arabic_legal_pipeline:Extracted 129 records from نظام الإثبات.json
2026-04-13 17:08:52 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الإجراءات الجزائية.jso
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الإجراءات الجزائية.json
2026-04-13 17:08:52 | INFO | Extracted 222 records from نظام الإجراءات الجزائية.json
INFO:arabic_legal_pipeline:Extracted 222 records from نظام الإجراءات الجزائية.json
2026-04-13 17:08:52 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الإفلاس.json
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الإفلاس.json
2026-04-13 17:08:53 | INFO | Extracted 231 records from نظام الإفلاس.json
INFO:arabic_legal_pipeline:Extracted 231 records from نظام الإفلاس.json
2026-04-13 17:08:53 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الامتياز التجاري.json
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الامتياز التجاري.json
2026-04-13 17:08:53 | INFO | Extracted 27 records from نظام الامتياز التجاري.json
INFO:arabic_legal_pipeline:Extracted 27 records from نظام الامتياز التجاري.json
2026-04-13 17:08:53 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التجارة الإلكترونية.js
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التجارة الإلكترونية.jso
2026-04-13 17:08:53 | INFO | Extracted 26 records from نظام التجارة الإلكترونية.json
INFO:arabic_legal_pipeline:Extracted 26 records from نظام التجارة الإلكترونية.json
2026-04-13 17:08:53 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التحكيم.json
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التحكيم.json
2026-04-13 17:08:53 | INFO | Extracted 58 records from نظام التحكيم.json
INFO:arabic_legal_pipeline:Extracted 58 records from نظام التحكيم.json
2026-04-13 17:08:53 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التكاليف القضائية.jso
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التكاليف القضائية.json
2026-04-13 17:08:53 | INFO | Extracted 23 records from نظام التكاليف القضائية.json
INFO:arabic_legal_pipeline:Extracted 23 records from نظام التكاليف القضائية.json
2026-04-13 17:08:53 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التنفيذ (المحدث).jso
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام التنفيذ (المحدث).json
2026-04-13 17:08:53 | INFO | Extracted 98 records from نظام التنفيذ (المحدث).json
INFO:arabic_legal_pipeline:Extracted 98 records from نظام التنفيذ (المحدث).json
2026-04-13 17:08:53 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام السجل التجاري.json
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام السجل التجاري.json
2026-04-13 17:08:53 | INFO | Extracted 29 records from نظام السجل التجاري.json
INFO:arabic_legal_pipeline:Extracted 29 records from نظام السجل التجاري.json
2026-04-13 17:08:53 | INFO | Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الشركات (الجديد 1443
INFO:arabic_legal_pipeline:Loading file: /tmp/arabic_legal_pipeline_2hioi8un/الانظمة/نظام الشركات (الجديد 1443).j
2026-04-13 17:08:53 | INFO | Extracted 281 records from نظام الشركات (الجديد 1443).json
```

Preprocess القضايا (Case layer + chunking + labels)

> SECOND STAGE — Case Data Processing

Phases implemented:

1. Data Cleaning & Normalization
2. Text Structuring & Segmentation
3. Legal Entity Recognition
4. Tokenization & Embedding Preparation
5. Dataset Balancing & Deduplication
6. Data Validation & Quality Assurance

PART 0 — Imports and Configuration

```
import pandas as pd
import numpy as np
import json
import re
import uuid
import logging
import random
from pathlib import Path
from collections import defaultdict
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from bs4 import BeautifulSoup
from transformers import AutoTokenizer
import torch
```

```
# CONFIGURATION

class PipelineConfig:
    """
    Global configuration for preprocessing pipeline.
    """
    # ------------------------------------------------------
    # Paths
    # ------------------------------------------------------


    INPUT_PATH = Path("/content")   # folder
    OUTPUT_PATH = Path("output")

    # create output folder
    OUTPUT_PATH.mkdir(exist_ok=True, parents=True)

    # Tokenization
    TOKENIZER_NAME = "aubmindlab/bert-base-arabertv02"

    MAX_SEQUENCE_LENGTH = 512

    TOKENIZATION_LEVEL = "segment"  # options: case / segment / sentence


    # Arabic normalization flags
    REMOVE_DIACRITICS = True
    NORMALIZE_ARABIC_NUMBERS = False


    # Deduplication
    NEAR_DUPLICATE_THRESHOLD = 0.92


    # Dataset Split
    TRAIN_RATIO = 0.70
    VAL_RATIO = 0.15
    TEST_RATIO = 0.15
    RANDOM_SEED = 42
```

```
# LOGGING CONFIGURATION
LOG_FILE = PipelineConfig.OUTPUT_PATH / "pipeline.log"

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    handlers=[
        logging.FileHandler(LOG_FILE, encoding="utf-8"),
        logging.StreamHandler()
    ]
)
```

```
logger = logging.getLogger(__name__)


# UNIFIED CASE SCHEMA
UNIFIED_SCHEMA = [
    "case_id",
    "judgment_number",
    "court_name",
    "city",
    "hijri_date",
    "judgment_text_raw",
    "appeal_text_raw",
    "full_case_text_raw",
    "judgment_text_clean",
    "appeal_text_clean",
    "full_case_text_clean",
    "source_file",
    "source_format"
]

# SECTION HEADINGS USED IN SEGMENTATION
SECTION_HEADINGS = [
    "الوقائع",
    "الأسباب",
    "الحكم",
    "الطلبات",
    "الدفع",
    "الرد",
    "منطوق الحكم"
]

# LEGAL ENTITY LABELS
ENTITY_LABELS = {
    "PLAINTIFF": "المدعي",
    "DEFENDANT": "المدعى عليه",
    "COURT": "المحكمة",
    "CITY": "المدينة",
    "CASE_NUMBER": "رقم القضية",
    "DATE": "التاريخ",
    "AMOUNT": "مبلغ",
    "LAW_NAME": "اسم نظام",
    "ARTICLE_NUMBER": "رقم مادة",
    "VERDICT": "منطوق الحكم"
}


# ARABIC NORMALIZATION REGEX
ARABIC_DIACRITICS = re.compile(r'[\u0617-\u061A\u064B-\u0652]')
ALEF_VARIANTS = re.compile(r'[إأآا]')
TATWEEL = re.compile(r'ـ+')


# UTILITY FUNCTIONS
def generate_uuid():
    """Generate unique ID"""
    return str(uuid.uuid4())


def save_json(data, path):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)


def save_jsonl(records, path):
    with open(path, "w", encoding="utf-8") as f:
        for r in records:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")
```

```
print(list(Path("/content").glob("*")))
```

```
[PosixPath('/content/.config'), PosixPath('/content/cases_6039174d_5200 (1).json'), PosixPath('/content/output'),
```

Step 1: Data Cleaning & Normalization

```
# PHASE 1 — DATA CLEANING & NORMALIZATION
def load_raw_cases(input_path):
    """
    Load raw case files from JSON / JSONL / CSV / XLSX.
    """
```

```
    records = []
    for file in input_path.glob("*"):
        suffix = file.suffix.lower()
        logger.info(f"Loading file: {file.name}")
        if suffix == ".json":
            with open(file, encoding="utf-8") as f:
                data = json.load(f)
                if isinstance(data, list):
                    for r in data:
                        r["__source_file"] = file.name
                        r["__source_format"] = "json"
                        records.append(r)
                else:
                    data["__source_file"] = file.name
                    data["__source_format"] = "json"
                    records.append(data)

        elif suffix == ".jsonl":
            with open(file, encoding="utf-8") as f:
                for line in f:
                    r = json.loads(line)
                    r["__source_file"] = file.name
                    r["__source_format"] = "jsonl"
                    records.append(r)
        elif suffix == ".csv":
            df = pd.read_csv(file)
            for _, row in df.iterrows():
                r = row.to_dict()
                r["__source_file"] = file.name
                r["__source_format"] = "csv"
                records.append(r)

        elif suffix in [".xlsx", ".xls"]:
            df = pd.read_excel(file)
            for _, row in df.iterrows():
                r = row.to_dict()
                r["__source_file"] = file.name
                r["__source_format"] = "xlsx"
                records.append(r)

    logger.info(f"Total raw records loaded: {len(records)}")

    return records
```

```
# SCHEMA NORMALIZATION
def normalize_case_schema(record):
    """
    Convert raw case record into unified schema.
    """

    case_id = record.get("id") or generate_uuid()

    judgment_text = record.get("judgmentText") or ""
    appeal_text = record.get("appealText") or ""

    full_text = (judgment_text or "") + "\n\n" + (appeal_text or "")

    return {
        "case_id": case_id,
        "judgment_number": record.get("judgmentNumber"),
        "court_name": record.get("court"),
        "city": record.get("city"),
        "hijri_date": record.get("hijriDate"),
        "judgment_text_raw": judgment_text,
        "appeal_text_raw": appeal_text,
        "full_case_text_raw": full_text,
        "judgment_text_clean": None,
        "appeal_text_clean": None,
        "full_case_text_clean": None,
        "source_file": record.get("__source_file"),
        "source_format": record.get("__source_format")
    }
```

```
# TEXT CLEANING
def remove_html(text):

    soup = BeautifulSoup(text, "html.parser")
    return soup.get_text()

def normalize_arabic(text):
    """
```

```
    Arabic normalization preserving legal meaning.
    """
    text = TATWEEL.sub("", text)
    if PipelineConfig.REMOVE_DIACRITICS:
        text = ARABIC_DIACRITICS.sub("", text)
    text = ALEF_VARIANTS.sub("ا", text)

    return text

def remove_page_numbers(text):
    """
    Remove standalone page numbers.
    """
    text = re.sub(r"\n?\s*\d+\s*\n", "\n", text)

    return text


def normalize_whitespace(text):
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"\n\s*\n+", "\n", text)

    return text.strip()

def remove_noise_symbols(text):
    """
    Remove noisy characters but keep legal punctuation.
    """
    text = re.sub(r"[^\w\s\u0600-\u06FF\.\,\:\;\(\)\-]", " ", text)

    return text

def clean_single_text(text):
    if not isinstance(text, str):
        return ""

    text = remove_html(text)
    text = remove_page_numbers(text)
    text = remove_noise_symbols(text)
    text = normalize_arabic(text)
    text = normalize_whitespace(text)

    return text
```

```
# CLEANING PIPELINE
def phase_1_data_cleaning():
    logger.info("Starting Phase 1 — Data Cleaning")
    raw_records = load_raw_cases(PipelineConfig.INPUT_PATH)
    normalized_records = []
    noise_report = []

    for r in raw_records:
        case = normalize_case_schema(r)
        j_raw = case["judgment_text_raw"]
        a_raw = case["appeal_text_raw"]
        j_clean = clean_single_text(j_raw)
        a_clean = clean_single_text(a_raw)
        case["judgment_text_clean"] = j_clean
        case["appeal_text_clean"] = a_clean
        case["full_case_text_clean"] = (j_clean or "") + "\n\n" + (a_clean or "")
        if len(j_raw) != len(j_clean):
            noise_report.append({
                "case_id": case["case_id"],
                "field": "judgment_text",
                "original_length": len(j_raw),
                "clean_length": len(j_clean)
            })

        normalized_records.append(case)

    df = pd.DataFrame(normalized_records)
    df = df[UNIFIED_SCHEMA]
    df.to_csv(
        PipelineConfig.OUTPUT_PATH / "02_cleaned_cases.csv",
        index=False,
        encoding="utf-8"
    )

    save_json(
        noise_report,
        PipelineConfig.OUTPUT_PATH / "cleaning_noise_report.json"
    )
```

```
    logger.info(f"Phase 1 complete — cleaned cases: {len(df)}")
    return df


    cleaned_df = phase_1_data_cleaning()
    print("Rows:", len(cleaned_df))
    cleaned_df.head()
```

```
cleaned_df = phase_1_data_cleaning()

print("Rows:", len(cleaned_df))
cleaned_df.head()
```

```
Rows: 4677
```

	case_id	judgment_number	court_name	city	hijri_date	judgment_text_ra
0	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	4630082310	المحكمة التجارية	الرياض	None	حمد لله والصلاة والسلام على رسول الله :أما ب
1	Gtwi21o7wSn-SSueeVqtaIZlTMnt0N4WymYtp1q4bc1OFp...	4630585068	المحكمة التجارية	الرياض	None	حمد لله والصلاة والسلام على رسول الله، أما ب
2	fkii3nB17yOw8j1iN0rixRxqxtLmi3Yh79c9nV-4KJlxXK...	4630106594	المحكمة التجارية	الرياض	None	حمد لله والصلاة والسلام على رسول الله، أما ب
3	MuO-5FiXHKDWEANdPqiMqtW2BCqmyrh3BeuNFoXZr-FsLv...	4630531487	المحكمة التجارية	الرياض	None	حمدلله والصلاة والسلام على رسول الله أما بعد
4	1FeVZrcOV2LCfbZ5NXMq3rskUxhBBLjpjDXCPGUocFRyKq...	4630454016	المحكمة التجارية	جدة	None	حمد لله والصلاة والسلام على رسول الله أما بع

Step 2: Text Structuring & Segmentation

```
# PHASE 2 — TEXT STRUCTURING & SEGMENTATION
def detect_section_heading(text):

    for heading in SECTION_HEADINGS:
        if heading in text:
            return heading

    return None

def assign_segment_label(section):
    mapping = {
        "الوقائع": "Fact",
        "الأسباب": "Reasoning",
        "الحكم": "Verdict",
        "الطلبات": "Request",
        "الدفع": "Procedural",
        "الرد": "Procedural",
        "منطوق الحكم": "Verdict"
    }

    return mapping.get(section, "Other")


def split_by_section(text):
    pattern = "(" + "|".join(SECTION_HEADINGS) + ")"
    parts = re.split(pattern, text)
    segments = []
    current_heading = None
    for part in parts:
        part = part.strip()
        if part in SECTION_HEADINGS:
            current_heading = part
            continue
        if part:
            segments.append((current_heading, part))
    return segments


def fallback_paragraph_split(text):
    paragraphs = re.split(r"\n+", text)
    segments = []
    for p in paragraphs:
        p = p.strip()
        if len(p) > 50:
            segments.append((None, p))
    return segments
```

```
def phase_2_segmentation(cleaned_df):
    logger.info("Starting Phase 2 — Segmentation")
    segments = []
    for _, row in cleaned_df.iterrows():
        case_id = row["case_id"]
        text = row["full_case_text_clean"]
        if not isinstance(text, str) or len(text) < 50:
            continue
        split_segments = split_by_section(text)
        if not split_segments:
            split_segments = fallback_paragraph_split(text)
        order = 0
        for section, seg_text in split_segments:
            order += 1
            segments.append({
                "segment_id": generate_uuid(),
                "case_id": case_id,
                "segment_order": order,
                "section_heading": section,
                "segment_text": seg_text,
                "segment_text_clean": clean_single_text(seg_text),
                "source_part": "combined",
                "segment_type_raw": assign_segment_label(section)
            })

    seg_df = pd.DataFrame(segments)

    seg_df.to_csv(
        PipelineConfig.OUTPUT_PATH / "03_segmented_cases.csv",
        index=False,
        encoding="utf-8"
    )

    save_jsonl(
        segments,
        PipelineConfig.OUTPUT_PATH / "03_segmented_cases.jsonl"
    )

    logger.info(f"Phase 2 complete — segments created: {len(seg_df)}")
    return seg_df
```

```
segmented_df = phase_2_segmentation(cleaned_df)
print("Segments:", len(segmented_df))
segmented_df.head()
```

```
Segments: 110568
```

	segment_id	case_id	segment_order	section_heading	segment_text	segm
0	92d6c5c5-1f67-4afe-97e4-b5794b4e69d2	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	1	None	الحمد لله والصلاة والسلام على رسول الله :اما ب...	م على
1	ba1da7b7-2ec2-426d-a65d-c86a2c6179e2	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	2	الوقائع	تتلخص وقائع هذه : القضية بالقدر اللازم للحكم ف...	ة :
2	fb8f8de7-4d9d-41a5-8ed9-f1f91a1ec7d4	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	3	الرد	عليها برقم.. الى المحكمة التجارية بالرياض بصحي...	حي

Step3: Legal Entity Recognition & Annotation:

> 3.1 Rule-based Entity Extraction ✔

> 3.2 AraBERT / AraLegal-BERT NER (placeholder)

> 3.3 Annotation dataset generation ✔

```
# PHASE 3 — LEGAL ENTITY RECOGNITION
import re
import pandas as pd

ENTITY_PATTERNS = {
    "وكيل_المدعي": r"وكيل\s+المدعي",
    "وكيل_المدعى_عليه": r"وكيل\s+المدعى عليه",
    "مبلغ": r"\d[\d,\.]*\s?د",
    "رقم_مادة": r"المادة\s*[\( ]?\s*[0-9٠١٢٣٤٥٦٧٨٩]+\s*[\) ]?",
    "اسم_نظام": r"(?:اللائحة\s+التنفيذية\s+لنظام\s+[\u0600-\u06FF\s]+|نظام\s+[\u0600-\u06FF\s]+)",
    "محكمة": r"المحكمة\s+[\u0600-\u06FF\s]+",
    "تاريخ": r"\d{1,2}/\d{1,2}/\d{4}"
}

PARTY_HEADERS = [
    ("المدعي", "مدعي"),
    ("المدعية", "مدعية"),
    ("المدعى عليه", "مدعى_عليه"),
    ("المدعى عليها", "مدعى_عليها"),
]

STOP_SECTION_HEADERS = {
    "الوقائع", "الطلبات", "الأسباب", "نص الحكم",
    "الحكم", "منطوق الحكم", "الموضوع"
}

def normalize_entity_text(text):
    if not text:
        return ""
    text = re.sub(r"\s+", " ", text).strip()
    text = text.lstrip(":  -- ").strip()
    return text

def is_stop_line(line):
    x = normalize_entity_text(line)
    if not x:
        return True
    if x in STOP_SECTION_HEADERS:
        return True
    if x.startswith("بموجب") or x.startswith("تقدّم") or x.startswith("تقدم"):
        return True
    if x.startswith("إلى المحكمة") or x.startswith("صحيفة دعوى"):
        return True
    return False

def looks_like_party_name(line):
    """
    Accept party-like lines, reject narrative lines.
    """
    x = normalize_entity_text(line)
    if not x:
        return False

    # reject if it is just another header or role word
    if x in STOP_SECTION_HEADERS:
        return False
    if x in {"المدعي", "المدعية", "المدعى عليه", "المدعى عليها"}:
        return False

    # reject obvious narrative beginnings
    bad_starts = [
        "بموجب", "تقدّم", "تقدم", "إلى المحكمة", "بصحيفة", "تضمنت",
        "وحيث", "وبعد", "وقد", "ثم", "أولاً", "ثانياً", "ثالثاً"
    ]
    if any(x.startswith(b) for b in bad_starts):
        return False

    # very short junk
    if len(x) < 3:
        return False

    return True

def extract_party_entities(text, case_id, segment_id):
    entities = []
    lines = text.splitlines()

    for i, line in enumerate(lines):
        current = normalize_entity_text(line)

        for label, header in PARTY_HEADERS:
            # match lines like:
```

```
            # المدعي
            # المدعي:
            # المدعي :
            if re.fullmatch(rf"{re.escape(header)}\s*[:  ]?", current):
                name_lines = []
                start_char = None

                # take only the next 1–2 meaningful lines max
                for j in range(i + 1, min(i + 4, len(lines))):
                    nxt = lines[j]
                    nxt_norm = normalize_entity_text(nxt)

                    if not nxt_norm:
                        continue

                    # stop if we hit another header/section
                    if any(re.fullmatch(rf"{re.escape(h)}\s*[:  ]?", nxt_norm) for _, h in PARTY_HEADERS):
                        break
                    if is_stop_line(nxt_norm):
                        break

                    # take first valid party-like line
                    if looks_like_party_name(nxt_norm):
                        if start_char is None:
                            # approximate char offset from original text
                            pos = text.find(nxt)
                            start_char = pos if pos != -1 else 0

                        name_lines.append(nxt_norm)

                        # if next line is just a continuation like "(شركة شخص واحد)"
                        # or a short descriptor, allow one extra line only
                        if j + 1 < len(lines):
                            nxt2 = normalize_entity_text(lines[j + 1])
                            if nxt2 and not is_stop_line(nxt2):
                                if (
                                    nxt2.startswith("(")
                                    or nxt2.startswith("شركة")
                                    or nxt2.startswith("مؤسسة")
                                    or "شخص واحد" in nxt2
                                ):
                                    name_lines.append(nxt2)
                        break

                entity_text = normalize_entity_text(" ".join(name_lines))

                if entity_text:
                    entities.append({
                        "entity_id": generate_uuid(),
                        "case_id": case_id,
                        "segment_id": segment_id,
                        "entity_text": entity_text,
                        "entity_label": label,
                        "start_char": start_char if start_char is not None else 0,
                        "end_char": (start_char if start_char is not None else 0) + len(entity_text),
                        "extraction_method": "rule_based",
                        "confidence": 0.9
                    })

    return entities

def extract_entities_from_segment(text, case_id, segment_id):
    entities = []

    # 1) party entities
    entities.extend(extract_party_entities(text, case_id, segment_id))

    # 2) other entities
    for label, pattern in ENTITY_PATTERNS.items():
        for m in re.finditer(pattern, text):
            entities.append({
                "entity_id": generate_uuid(),
                "case_id": case_id,
                "segment_id": segment_id,
                "entity_text": m.group().strip(),
                "entity_label": label,
                "start_char": m.start(),
                "end_char": m.end(),
                "extraction_method": "rule_based",
                "confidence": 0.9
            })
```

```
    return entities

def phase_3_entity_extraction(segmented_df):
    logger.info("Starting Phase 3 — Entity Extraction")
    all_entities = []
    annotation_records = []

    for _, row in segmented_df.iterrows():
        case_id = row["case_id"]
        segment_id = row["segment_id"]
        text = row["segment_text_clean"]

        entities = extract_entities_from_segment(text, case_id, segment_id)
        all_entities.extend(entities)

        annotation_records.append({
            "case_id": case_id,
            "segment_id": segment_id,
            "text": text,
            "entities": entities
        })

    entities_df = pd.DataFrame(all_entities)

    entities_df.to_csv(
        PipelineConfig.OUTPUT_PATH / "04_extracted_entities.csv",
        index=False,
        encoding="utf-8"
    )

    save_jsonl(
        annotation_records,
        PipelineConfig.OUTPUT_PATH / "14_annotation_ready.jsonl"
    )

    logger.info(f"Phase 3 complete — entities extracted: {len(entities_df)}")
    return entities_df
```

```
entities_df = phase_3_entity_extraction(segmented_df)
print("Entities:", len(entities_df))
entities_df.head()
```

```
Entities: 131126
```

	entity_id	case_id	segment_id	entity_text	entity_label	start_char
0	fb61787b-5ae3-452a-bdf0-e9939ec3dd3b	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	fb8f8de7-4d9d-41a5-8ed9-f1f91a1ec7d4	٦٣٢٥٢٠ريال	مبلغ	428
1	e4f54416-deae-440b-be1f-5b351fc46210	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	fb8f8de7-4d9d-41a5-8ed9-f1f91a1ec7d4	١٠٩١٠٩.٧٠ريال	مبلغ	538
2	598da842-c197-406a-8843-77c8abfa9e66	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	fb8f8de7-4d9d-41a5-8ed9-f1f91a1ec7d4	١٠٩١٠٩.٧٠ريال	مبلغ	1452

```
entities_df[entities_df["entity_label"].isin(["مدعي", "مدعية", "مدعى_عليه", "مدعى_عليها"])].head(20)
```

entity_id	case_id	segment_id	entity_text	entity_label	start_char	end_char	extraction_method	confidence

Step 4 — Tokenization + Embeddings

```
# PHASE 4 — TOKENIZATION & EMBEDDING PREPARATION
from transformers import AutoTokenizer
import torch

def load_tokenizer():
    logger.info("Loading tokenizer")
    tokenizer = AutoTokenizer.from_pretrained(
        PipelineConfig.TOKENIZER_NAME
    )

    return tokenizer

def tokenize_segment(tokenizer, text):
    tokens = tokenizer(
        text,
        padding="max_length",
        truncation=True,
        max_length=PipelineConfig.MAX_SEQUENCE_LENGTH,
        return_tensors="pt"
    )
    return {
        "input_ids": tokens["input_ids"].squeeze().tolist(),
        "attention_mask": tokens["attention_mask"].squeeze().tolist(),
        "token_type_ids": tokens.get("token_type_ids", torch.zeros_like(tokens["input_ids"])).squeeze().tolist()
    }


def phase_4_tokenization(segmented_df):
    logger.info("Starting Phase 4 — Tokenization")
    tokenizer = load_tokenizer()
    tokenized_records = []
    for _, row in segmented_df.iterrows():
        case_id = row["case_id"]
        segment_id = row["segment_id"]
        text = row["segment_text_clean"]
        tokens = tokenize_segment(tokenizer, text)
        tokenized_records.append({
            "case_id": case_id,
            "segment_id": segment_id,
            "input_ids": tokens["input_ids"],
            "attention_mask": tokens["attention_mask"],
            "token_type_ids": tokens["token_type_ids"],
            "token_count": len(tokens["input_ids"])
        })

    token_df = pd.DataFrame(tokenized_records)
    token_df.to_csv(
        PipelineConfig.OUTPUT_PATH / "05_tokenized_metadata.csv",
        index=False,
        encoding="utf-8"
    )

    logger.info(f"Phase 4 complete — tokenized segments: {len(token_df)}")
    return token_df
```

```
token_df = phase_4_tokenization(segmented_df)
print("Tokenized segments:", len(token_df))
token_df.head()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

```
config.json: 100%                                   384/384 [00:00<00:00, 19.7kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
tokenizer_config.json: 100%                         381/381 [00:00<00:00, 13.2kB/s]
vocab.txt:       825k/? [00:00<00:00, 14.5MB/s]
tokenizer.json:       2.64M/? [00:00<00:00, 52.2MB/s]
special_tokens_map.json: 100%                       112/112 [00:00<00:00, 1.85kB/s]
Tokenized segments: 110568
```

	case_id	segment_id	input_ids	attention_mask	token_type_ids	token_c
0	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	1ff5961f-8597-492e-af93-54e5811d4738	[2, 6620, 3074, 36829, 6404, 323, 9098, 647, 3...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	
1	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	611435fa-beda-466d-b9d1-7a069ee8b382	[2, 31, 31174, 12369, 450, 2521, 31477, 8114, ...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	

Step 5 — Dataset Balancing and Deduplication

```
# PHASE 5 — DATASET BALANCING & DEDUPLICATION
from sklearn.model_selection import train_test_split
def remove_exact_duplicates(cleaned_df):
    logger.info("Removing exact duplicate cases")
    before = len(cleaned_df)
    dedup_df = cleaned_df.drop_duplicates(
        subset=["judgment_number"]
    )

    after = len(dedup_df)
    duplicates_removed = before - after
    logger.info(f"Duplicates removed: {duplicates_removed}")
    dup_report = cleaned_df[
        cleaned_df.duplicated(subset=["judgment_number"], keep=False)
    ]

    dup_report.to_csv(
        PipelineConfig.OUTPUT_PATH / "09_duplicates_removed.csv",
        index=False,
        encoding="utf-8"
    )
    return dedup_df


def split_dataset_by_case(segmented_df):
    logger.info("Splitting dataset by case_id")
    case_ids = segmented_df["case_id"].unique()
    train_cases, temp_cases = train_test_split(
        case_ids,
        test_size=0.30,
        random_state=42
    )

    val_cases, test_cases = train_test_split(
        temp_cases,
        test_size=0.50,
```

```
            random_state=42
        )

        train_df = segmented_df[segmented_df["case_id"].isin(train_cases)]
        val_df = segmented_df[segmented_df["case_id"].isin(val_cases)]
        test_df = segmented_df[segmented_df["case_id"].isin(test_cases)]

        train_df.to_csv(
            PipelineConfig.OUTPUT_PATH / "06_train.csv",
            index=False,
            encoding="utf-8"
        )

        val_df.to_csv(
            PipelineConfig.OUTPUT_PATH / "07_val.csv",
            index=False,
            encoding="utf-8"
        )

        test_df.to_csv(
            PipelineConfig.OUTPUT_PATH / "08_test.csv",
            index=False,
            encoding="utf-8"
        )

        logger.info(
            f"Split complete — train: {len(train_df)}, val: {len(val_df)}, test: {len(test_df)}"
        )

        return train_df, val_df, test_df


    def generate_balance_report(train_df, val_df, test_df):
        report = {
            "train_segments": len(train_df),
            "val_segments": len(val_df),
            "test_segments": len(test_df),

            "train_cases": train_df["case_id"].nunique(),
            "val_cases": val_df["case_id"].nunique(),
            "test_cases": test_df["case_id"].nunique()
        }

        save_json(
            report,
            PipelineConfig.OUTPUT_PATH / "12_balance_report.json"
        )

        return report

    def phase_5_dataset_preparation(cleaned_df, segmented_df):
        logger.info("Starting Phase 5 — Dataset preparation")
        dedup_df = remove_exact_duplicates(cleaned_df)
        train_df, val_df, test_df = split_dataset_by_case(segmented_df)
        report = generate_balance_report(train_df, val_df, test_df)
        logger.info("Phase 5 complete")
        return train_df, val_df, test_df, report
```

```
    train_df, val_df, test_df, balance_report = phase_5_dataset_preparation(
        cleaned_df,
        segmented_df
    )

    balance_report
```

```
    {'train_segments': 77111,
     'val_segments': 16809,
     'test_segments': 16648,
     'train_cases': 3273,
     'val_cases': 702,
     'test_cases': 702}
```

Step 6 — Data Validation and Quality Assurance

> Step 6 — Part 1: Structural Validation

> Step 6 - Part 2: Linguistic Validation

> Step 6 — Part 3: Legal Validation

> Step 6 — Part 4: Logging and Traceability System

Step 6 — Part 5: Model-Ready Data Approval

```
# PHASE 6 — DATA VALIDATION & QUALITY ASSURANCE
from datetime import datetime
import random


# 6.1 STRUCTURAL VALIDATION
def structural_validation(cleaned_df, segmented_df, entities_df):
    logger.info("Running Structural Validation")
    issues = []
    # empty text check
    empty_segments = segmented_df[
        segmented_df["segment_text_clean"].isna() |
        (segmented_df["segment_text_clean"].str.strip() == "")
    ]

    for _, row in empty_segments.iterrows():
        issues.append({
            "issue_type": "empty_segment",
            "case_id": row["case_id"],
            "segment_id": row["segment_id"]
        })

    # missing ids
    missing_ids = segmented_df[
        segmented_df["segment_id"].isna() |
        segmented_df["case_id"].isna()
    ]
    for _, row in missing_ids.iterrows():
        issues.append({
            "issue_type": "missing_ids",
            "case_id": row.get("case_id"),
            "segment_id": row.get("segment_id")
        })
    logger.info(f"Structural issues detected: {len(issues)}")
    return issues


# 6.2 LINGUISTIC VALIDATION
CRITICAL_LEGAL_TERMS = [
    "المدعي",
    "المدعى عليه",
    "المحكمة",
    "الحكم",
    "الوقائع",
    "الأسباب",
    "المادة",
    "النظام"
]

def linguistic_validation(segmented_df):
    logger.info("Running Linguistic Validation")
    warnings = []
    for _, row in segmented_df.iterrows():
        text = row["segment_text_clean"]
        if text is None:
            continue
        contains_term = any(term in text for term in CRITICAL_LEGAL_TERMS)
        if contains_term:
            # if term present but entity not extracted
            warnings.append({
                "issue_type": "possible_missing_entity",
                "case_id": row["case_id"],
                "segment_id": row["segment_id"],
                "text_sample": text[:200]
            })

    logger.info(f"Linguistic warnings: {len(warnings)}")
    return warnings


# 6.3 LEGAL VALIDATION
LEGAL_CITATION_PATTERN = re.compile(r"(المادة\s*\(?\d+\)?|نظام\s+\w+)")
def legal_validation(segmented_df):
    logger.info("Running Legal Validation")
    findings = []
    for _, row in segmented_df.iterrows():
        text = row["segment_text_clean"]
        if text is None:
            continue
```

```
        matches = LEGAL_CITATION_PATTERN.findall(text)
        for m in matches:
            findings.append({
                "case_id": row["case_id"],
                "segment_id": row["segment_id"],
                "citation": m
            })

    logger.info(f"Legal citations detected: {len(findings)}")
    return findings


# 6.4 LOGGING & TRACEABILITY
def generate_processing_log(stats):
    log_data = {
        "timestamp": datetime.utcnow().isoformat(),
        "records_loaded": stats["records_loaded"],
        "cases_total": stats["cases_total"],
        "segments_generated": stats["segments_generated"],
        "entities_extracted": stats["entities_extracted"],
        "duplicates_removed": stats["duplicates_removed"],
        "validation_warnings": stats["validation_warnings"]
    }

    save_json(
        log_data,
        PipelineConfig.OUTPUT_PATH / "13_processing_log.json"
    )

    return log_data


# 6.5 MODEL READY APPROVAL
def generate_model_ready_manifest(
    total_cases,
    total_segments,
    total_entities
):

    manifest = {
        "dataset_name": "Saudi_Commercial_Cases",
        "cases": total_cases,
        "segments": total_segments,
        "entities": total_entities,
        "language": "Arabic",
        "status": "APPROVED_FOR_MODEL_TRAINING"
    }

    save_json(
        manifest,
        PipelineConfig.OUTPUT_PATH / "15_model_ready_manifest.json"
    )
    return manifest


# MASTER VALIDATION PIPELINE
def phase_6_validation(cleaned_df, segmented_df, entities_df):
    logger.info("Starting Phase 6 — Validation & QA")
    structural_issues = structural_validation(
        cleaned_df,
        segmented_df,
        entities_df
    )

    linguistic_warnings = linguistic_validation(
        segmented_df
    )

    legal_findings = legal_validation(
        segmented_df
    )

    validation_report = {
        "structural_issues": len(structural_issues),
        "linguistic_warnings": len(linguistic_warnings),
        "legal_citations_detected": len(legal_findings)
    }

    save_json(
        validation_report,
        PipelineConfig.OUTPUT_PATH / "11_validation_report.json"
    )
```

```
    problematic = structural_issues + linguistic_warnings

    pd.DataFrame(problematic).to_csv(
        PipelineConfig.OUTPUT_PATH / "10_problematic_records.csv",
        index=False,
        encoding="utf-8"
    )

    logger.info("Phase 6 completed")
    return validation_report


# MAIN PIPELINE EXECUTION
def main():
    logger.info("Starting Saudi Legal Case Preprocessing Pipeline")

    # Phase 1
    raw_df = load_raw_cases(PipelineConfig.INPUT_PATH)
    cleaned_df = phase_1_data_cleaning()

    # Phase 2
    segmented_df = phase_2_segmentation(cleaned_df)

    # Phase 3
    entities_df = phase_3_entity_extraction(segmented_df)

    # Phase 4
    tokenized_df = phase_4_tokenization(segmented_df)

    # Phase 5
    train_df, val_df, test_df, balance_report = phase_5_dataset_preparation(
        cleaned_df,
        segmented_df
    )

    # Phase 6
    validation_report = phase_6_validation(
        cleaned_df,
        segmented_df,
        entities_df
    )

    # Logging
    stats = {
        "records_loaded": len(raw_df),
        "cases_total": cleaned_df["case_id"].nunique(),
        "segments_generated": len(segmented_df),
        "entities_extracted": len(entities_df),
        "duplicates_removed": 0,
        "validation_warnings": validation_report["linguistic_warnings"]
    }

    generate_processing_log(stats)
    generate_model_ready_manifest(
        total_cases=cleaned_df["case_id"].nunique(),
        total_segments=len(segmented_df),
        total_entities=len(entities_df)
    )

    logger.info("Pipeline finished successfully")

if __name__ == "__main__":
```

```
/tmp/ipykernel_8244/2842926740.py:95: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
  "timestamp": datetime.utcnow().isoformat(),
```

```
from google.colab import drive
drive.mount('/content/drive')
```